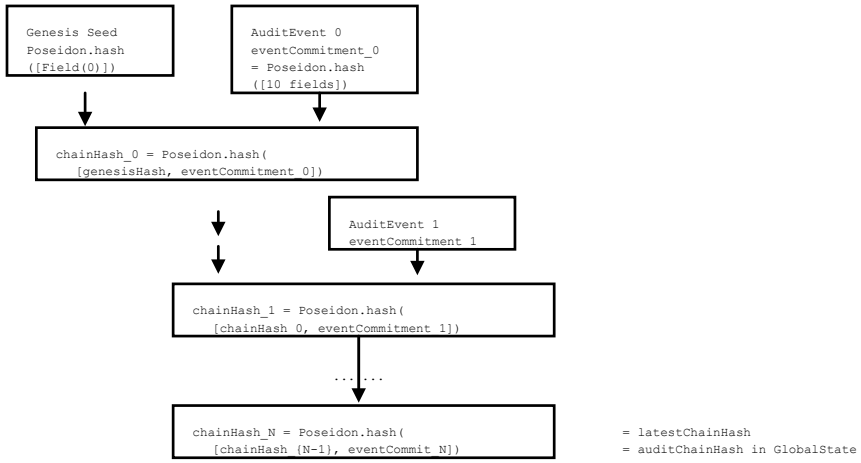
FIG. 10A -- POSEIDON HASH AUDIT CHAIN: HASH CHAIN LINKING EVENT COMMITMENT 1000

SECTION A: HASH CHAIN LINKING 1000

Genesis: Poseidon.hash([Field(0)])
Rule: chainHash_n = Poseidon.hash([chainHash_{n-1}, eventCommitment_n])



TAMPER DETECTION: If event K is modified -> eventCommitment_K changes -> chainHash_K changes
-> ALL subsequent chainHash values break. Verification walks forward from genesis.

SECTION B: TEN-FIELD EVENT COMMITMENT 1002

```
eventCommitment = Poseidon.hash([
   F1: walletId                      wallet involved
   F2: oldRegistryRoot               registry root BEFORE
   F3: newRegistryRoot               registry root AFTER
   F4: oldLifecycleRoot              lifecycle root BEFORE
   F5: newLifecycleRoot              lifecycle root AFTER
   F6: sequenceNumber                monotonic event sequence
   F7: isRegistration                1 if wallet reg, else 0
   F8: ndcHash                       drug identity (0 if non-claim)
   F9: patientPayCents               patient copay (0 if non-claim)
   F10: planPayCents                 insurer pay (0 if non-claim)
])
```

Registration events: F8-F10 = 0. Claim events: F8-F10 populated from proof output.

AuditEvent Structure:

```
CORE IDENTITY:
   sequenceNumber (monotonic from L2Receipt)
   receiptId (links to L2Receipt)
   walletId (opaque Poseidon hash)
   isRegistration (boolean)

STATE ROOTS (before/after):
   oldRegistryRoot, newRegistryRoot
   oldLifecycleRoot, newLifecycleRoot

CLAIM DATA (optional):
   ndcHash?, patientPayCents?, planPayCents?
```

```
CRYPTOGRAPHIC CHAIN:
   eventCommitment = Poseidon.hash([10 fields])
   chainHash = Poseidon.hash([prev, commit])

METADATA:
   timestamp (ms epoch)
   l1Status ('pending'|'batched'|'settled')
   l2BlockNumber? (set after batching)
   l1BatchNumber? (set after batching)
```